OAuth 2.0 vs OAuth 2.1 – Analyse des failles de sécurité

Contexte:

Un partenaire a identifié que l'utilisation du grant_type=password dans OAuth est une méthode dépréciée représentant une faille de sécurité.

1. Password Grant (grant_type=password)

- **OAuth 2.0:** autorisé mais fortement déconseillé
- **OAuth 2.1:** supprimé

Le client collecte directement le login et le mot de passe de l'utilisateur et les envoie au serveur OAuth. Sa représente une faille car le client ne doit jamais connaître le mot de passe.

Risque de fuite dans les logs

Les identifiants peuvent être enregistrés dans les logs (API Gateway, reverse proxy). Toute fuite de logs entraîne une fuite de mots de passe.

Cas le plus critique

Un attaquant accède aux logs et récupère des identifiants utilisateurs, entraînant une compromission totale des comptes.

2. Implicit Grant

- **OAuth 2.0:** Autorisé
- **OAuth 2.1:** Supprimé

C'est dangereux car le token d'accès est visible dans l'URL (#fragment) et il est peut-etre lu dans l'historique du navigateur.

Cas critique

Un token est récupéré via l'historique navigateur ou un script tiers.

3. Authorization Code sans PKCE (Proof Key for Code Exchange)

Sans PKCE, le code OAuth peut être intercepté.
PKCE est obligatoire en OAuth 2.1. Dans OAuth 2.0 le PKCE est optionnel

Cas critique

Dans OAuth 2.0 le PKCE est optionnel ce qui est un grand risque car un attaquant peut intercepté le code et obtenir un token valide.

4. Les Points “Chauds” dans le système du partenaire.

Password Grant est encore utilisé alors qu’il représente un risque d’exposion des données.

5. Conclusion

OAuth 2.1 corrige ces failles en supprimant les mécanismes dangereux et en imposant les bonnes pratiques.